

Intra Projects get_next_line Edit

4–5 minutes

Preliminaries

The review is done in the presence of the evaluated learner. This is how everyone progresses: by interacting with others.

- As soon as an exercise is non-functional, the review stops. You can look at the code of the following exercises, but they will not be graded.

Yes No

README file

Does the repository contain a README.md file at its root, and does it include all of the following?

- The first line is italicized and formatted exactly as: *This project has been created as part of the 42 curriculum by <login1>[, <login2>[, <login3>[...]]]*.
- A "Description" section explaining the project's purpose and providing a brief overview.
- An "Instructions" section with relevant details about compilation, installation, and/or execution.
- A "Resources" section listing references (documentation, tutorials, etc.) and explaining how AI was used, specifying for which tasks and which parts of the project.

- A detailed description of the library created for the project.
- A detailed explanation and justification of the algorithm selected for the project.

If any of the required elements is missing, the grade is 0.

Yes No

Norminette

Run the Norminette. If there is an error, the evaluation stops here.

You can keep going and discuss the implementation of the code, but the

assignment will not be graded.

Yes No

Compilation

It must be possible to compile the project with the usual flags, and with the

-D BUFFER_SIZE as well.

The project must compile successfully both with and without this flag, in addition to the

following flags: -Wall -Wextra -Werror.

If there is an error, the evaluation stops here.

You can keep going and discuss the implementation of the code, but the

assignment will not be graded.

Yes No

Error management

Carry out AT LEAST the following tests to try to stress the error

management.

- Pass an arbitrary, invalid file descriptor (e.g., 42) to `get_next_line`. The function must return `NULL`.
- Check the error returns for `read` and `malloc`. If there is an error, the evaluation stops here. You can keep going and discuss the implementation of the code, but the assignment will not be graded.

Yes No

Testing

As the evaluator, you are expected to provide a main which will always check:

- The return value of the `get_next_line` is `NULL` in case of error.
- Otherwise, the function returns the line read, always with a `\n` at the end except if the end of file was reached and does not end with a `\n` character.

Test all the possible combinations of the following rules:

- Large `BUFFER_SIZE` (>1024)
- Small `BUFFER_SIZE` (< 8 , and 1)
- `BUFFER_SIZE` exactly the length of the line to read
- 1 byte variant ($\pm$) between the line and the `BUFFER_SIZE`
- Read on `stdin`
- Read from a file
- (Multiple/Single) Long line ($2k+$ characters)
- (Multiple/Single) Short line (< 4 characters, even 1)
- (Multiple/Single) Empty line

These tests should allow you to assess the robustness of the

student's

get_next_line implementation.

If there is an error, the evaluation stops here.

Yes No

Quick live coding modification

Please ask the reviewee to modify the project code to support the use of a custom separator character, as chosen by the reviewer. This should not be limited to a single function but applied wherever relevant in the project.

The reviewee must then demonstrate that the modification works correctly using concrete examples.

The entire task, including the demonstration, should take no more than 10 minutes.

Was this procedure followed and did everything work correctly?

Yes No

Multiple fd reading

Perform the same tests as before, but this time, launch multiple instances

of get_next_line with different file descriptor.

Make sure that each get_next_line is returning the correct line.

Combine with a non-existing fd to check for errors.

Yes No

Single static variable

Review the code to confirm that only one static variable is used.

Award the point accordingly.

Yes No